

# Multi-Client TCP Chat Server & Performance Report

Network Assignment 5 • Mininet Simulation & Traffic Analysis



**Student Name:**  
Munmi Das



**Date:**  
1/08/2026



**Topology:**  
Mininet Single Switch,  
5 Hosts (h1-h5)



**Protocol:**  
TCP / IP (Port 5000)

## 1. Objective

The objective of this assignment is to extend the multi-client chat application from Assignment 4 into a robust, concurrent TCP networking system deployed within a Mininet simulation environment. Key requirements include implementing enhanced client management, private messaging, online user listing, persistent chat history buffering, and thorough performance evaluation through CSV logs, graphs, and Wireshark packet inspection.

## 2. System Architecture

The architecture is structured around a central server running on host h1 (IP: 10.0.0.1) and multiple client nodes (h2 through h5) running `client.py`. Communication relies on reliable TCP sockets over port 5000, managed asynchronously using Python's threading primitives.

## 3. Design Improvements over Assignment 4

- **Concurrency & Threading:** Upgraded from single-threaded polling to dedicated daemon threads per client connection to eliminate deadlocks.
- **Private Messaging Engine:** Added command parsing for `/msg <user> <text>` routed securely through the server.
- **History Buffering:** Implemented a persistent queue storing the last 5 messages delivered to newly joining clients.
- **Quantitative Logging:** Automated CSV output generation capturing exact performance metrics (delay and throughput).

## 4. Program Design

`server.py` maintains an active client dictionary mapping usernames to socket objects. It intercepts commands (`/list`, `/msg`, `/history`) and broadcasts standard chat frames. `client.py` operates with dual threads: one handling user stdin input and another listening for incoming server broadcast streams.

## 5. Experimental Setup

Simulated within Ubuntu via Mininet using a single OpenFlow switch and 5 hosts. Automated testing scripts injected variable client loads (2, 3, and 4 concurrent clients) while capturing throughput and latency metrics into `performance_results.csv` as recorded during the test runs.

## 6. Results & Performance CSV Data

The performance metrics captured during the evaluation runs match the exact recorded CSV dataset:

### PERFORMANCE\_RESULTS.CSV (EXACT FORMAT)

```
clients,broadcast_messages,private_messages,avg_delay_ms,throughput_msgs_per_sec
2,30,20,12.5,45.0
```

```
3,30,20,18.0,38.5
4,30,20,24.0,31.5
```

Tabular breakdown of the performance metrics:

| Active Clients | Broadcast Msgs | Private Msgs | Avg Delay (ms) | Throughput (msgs/sec) |
|----------------|----------------|--------------|----------------|-----------------------|
| 2 Clients      | 30             | 20           | 12.5 ms        | 45.0 msgs/sec         |
| 3 Clients      | 30             | 20           | 18.0 ms        | 38.5 msgs/sec         |
| 4 Clients      | 30             | 20           | 24.0 ms        | 31.5 msgs/sec         |

Tabular breakdown of the performance metrics:

| Active Clients | Broadcast Msgs | Private Msgs | Avg Delay (ms) | Throughput (msgs/sec) |
|----------------|----------------|--------------|----------------|-----------------------|
| 2 Clients      | 30             | 20           | 12.5 ms        | 45.0 msgs/sec         |
| 3 Clients      | 30             | 20           | 18.0 ms        | 38.5 msgs/sec         |
| 4 Clients      | 30             | 20           | 24.0 ms        | 31.5 msgs/sec         |

## 7. Graph Analysis

- **Number of Clients vs Average Delivery Time:** Shows a near-linear growth in average delivery delay from 12.5 ms at 2 clients to 24.0 ms at 4 clients due to serialization wait times on the server socket buffer.
- **Number of Clients vs Throughput:** Demonstrates a gradual decline in aggregate throughput from 45.0 to 31.5 msgs/sec under increasing client concurrency.
- **Broadcast Messages vs Private Messages:** Highlights a balanced interaction ratio with 30 broadcast messages and 20 private messages.

## 8. Wireshark Verification

Packet analysis of `chat_traffic.pcapng` verified standard TCP 3-way handshakes, persistent ACK segments, and payload encapsulation containing plain-text chat frames on port 5000 without packet drops.

## 9. Reflection Answers

### (1) Why is server-side routing necessary for private messaging?

Server-side routing ensures security, centralized user registry lookup, and decoupling so clients do not need direct public IP visibility of each other.

### (2) How does maintaining user state improve the application?

It enables accurate delivery tracking, instant `/list` directory queries, and graceful socket cleanup upon unexpected disconnects.

### (3) How would the application scale to 100 users?

A standard multi-threaded model would exhaust OS file descriptors. Scaling to 100+ users requires migrating to an asynchronous I/O model using Python's `asyncio` or `selectors` module.

### (4) Which Wireshark packets verified private messaging?

TCP segments containing targeted application payload strings routed from the sender through server IP `10.0.0.1` to the specific receiver port.

### (5) What improvements would you implement next?

TLS encryption for secure payloads, persistent SQLite database storage for chat history, and a graphical user interface (GUI).

## 7. Conclusion

---

The multi-client TCP chat application was successfully deployed and verified in the Mininet simulation environment. Performance analysis validates that the system maintains stable delivery times and robust handling of both broadcast and private message traffic under concurrent client loads.